

Implement SSH, firewall, automation and SELinux on Linux Server TASK – 5

- 1. Enable and configure SSH on your Linux server so that only key-based authentication is allowed and password authentication is disabled.**

Ans :

- Install SSH server
- `sudo yum install openssh-server -y`
- Start and enable SSH service
- `sudo systemctl start sshd`
- `sudo systemctl enable sshd`
- Generate SSH key (on client machine)
- `ssh-keygen -t rsa`
- Copy public key to server
- `ssh-copy-id user@server_ip`
- Edit SSH configuration file
- `sudo nano /etc/ssh/sshd_config`
- Set the following parameters:
- `PasswordAuthentication no`
- `PubkeyAuthentication yes`
- `ChallengeResponseAuthentication no`
- `PermitRootLogin prohibit-password`
- Restart SSH service

- `sudo systemctl restart sshd`
 - Verify SSH configuration
 - `sudo sshd -T | grep passwordauthentication`
 - `sudo sshd -T | grep pubkeyauthentication`
 - Test login from client
 - `ssh user@server_ip`
- 2. Set up a basic firewall using UFW or firewalld to allow only SSH (port 22) and HTTP (port 80) traffic, and block all other incoming connections.**
Hint: After applying rules, use a port scanning tool like nmap from another machine to verify only the allowed ports are open.

Ans :

- `sudo systemctl enable firewalld`
 - `sudo systemctl start firewalld`
 - `sudo firewall-cmd --set-default-zone=public`
 - `sudo firewall-cmd --permanent --add-service=ssh`
 - `sudo firewall-cmd --permanent --add-service=http`
 - `sudo firewall-cmd --reload`
 - `sudo firewall-cmd --list-all`
- 3. Write a shell script automate_backup.sh that creates a backup of your /home directory and saves it with a timestamp in /backups.**

Ans :

- `#!/bin/bash`
- `# automate_backup.sh`
- `# Script to back up /home directory with timestamp`

```
SOURCE="/home"
DEST="/backups"
```

- `# Create backup directory if it doesn't exist`
`mkdir -p "$DEST"`
- `# Generate timestamp (YYYY-MM-DD_HH-MM-SS)`
`TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")`
- `# Backup filename`
`BACKUP_FILE="home_backup_${TIMESTAMP}.tar.gz"`
- `# Create compressed archive`
`tar -czf "$DEST/$BACKUP_FILE" "$SOURCE"`
- `# Check if backup was successful`
`if [ $? -eq 0 ]; then`
 `echo "Backup successful: $DEST/$BACKUP_FILE"`
 `else`
 `echo "Backup failed!"`
 `exit 1`
 `fi`

- 4. Check the current status of SELinux on your server, switch it to enforcing mode if it is not already, and verify the change using appropriate commands.**
Hint: Use the commands `getenforce` and `setenforce`.

Ans :

- `getenforce`
- `sestatus`
- `sudo setenforce 1`
- `getenforce`
- `sestatus`
- `sudo nano /etc/selinux/config`
- `# set: SELINUX=enforcing`
- `sudo reboot`

5. Use ChatGPT or Copilot to generate a checklist of at least 5 best practices for securing a Linux server, and compare it with your own server configuration to identify any missing steps.

Ans :

Security Area	Typical Default Setup	Best Practice Gap
System updates	Manual updates only	No auto security patching
SSH security	Password login enabled	No key-based auth, root login sometimes enabled
Firewall	Often inactive	No default deny rules
Brute-force protection	Not installed	Missing fail2ban
User management	Few users but broad sudo rights	Overprivileged accounts
Logging	Basic logs only	No monitoring/alerting

Security Area	Typical Default Setup	Best Practice Gap
Services	Multiple default services running	Unused services still enabled
Backups	Manual or none	No automated/offsite backups